3 A program reads data from the user and stores the data that is valid in a linear queue.

The queue is stored as a global 1D array, `QueueData`, of string values. The array needs space for 20 elements.

The global variable `QueueHead` stores the index of the first element in the queue.
The global variable `QueueTail` stores the index of the last element in the queue.

(a) The main program initialises all the elements in `QueueData` to a suitable null value, `QueueHead` to −1 and `QueueTail` to −1.

Write program code for the main program.

Save your program as **Question3_J24**. Copy and paste the program code into part **3(a)** in the evidence document.

[1]

(b) The function `Enqueue()` takes the data to insert into the queue as a parameter.

If the queue is **not** full, it inserts the parameter in the queue, updates the appropriate pointer(s) and returns `TRUE`. If the queue is full, it returns `FALSE`.

Write program code for `Enqueue()`.

Save your program. Copy and paste the program code into part **3(b)** in the evidence document.

[4]

(c) The function `Dequeue()` returns `"false"` if the queue is empty. If the queue is **not** empty, it returns the next item in the queue and updates the appropriate pointer(s).

Write program code for `Dequeue()`.

Save your program. Copy and paste the program code into part **3(c)** in the evidence document.

[3]

(d) The string values to be stored in the queue are 7 characters long. The first 6 characters are digits and the 7th character is a check digit. The check digit is calculated from the first 6 digits using this algorithm:

- multiply the digits in position 0, position 2 and position 4 by 1
- multiply the digits in position 1, position 3 and position 5 by 3
- calculate the sum of the products (add together the results from all of the multiplications)
- divide the sum of the products by 10 and round the result down to the nearest integer to get the check digit
- if the check digit equals 10 then it is replaced with '`X`'.

Example:

Data is 954123

Character position	**0**	**1**	**2**	**3**	**4**	**5**
Digit	9	5	4	1	2	3
Multiplier	1	3	1	3	1	3
Product	9	15	4	3	2	9

Sum of products = 9 + 15 + 4 + 3 + 2 + 9 = 42

Divide sum of products by 10: 42 / 10 = 4 (rounded down)

The check digit = 4. This is inserted into character position 6.

The data including the check digit is: **9541234**

A 7-character string is valid if the 7th character matches the check digit for that data. For example, the data 9541235 is invalid because the 7th character (5) does **not** match the check digit for 954123.

(i) The subroutine `StoreItems()` takes ten 7-character strings as input from the user and uses the check digit to validate each input.

Each valid input has the check digit removed and is stored in the queue using `Enqueue()`.
An appropriate message is output if the item is inserted. An appropriate message is output if the queue is already full.

Invalid inputs are **not** stored in the queue.

The subroutine counts and outputs the number of invalid items that were entered.

`StoreItems()` can be a procedure or a function as appropriate.

Write program code for `StoreItems()`.

Save your program.

Copy and paste the program code into part **3(d)(i)** in the evidence document.

[6]

(ii) Write program code to amend the main program to:

- call `StoreItems()`
- call `Dequeue()`
- output a suitable message if the queue was empty
- output the returned value if the queue was **not** empty.

Save your program.

Copy and paste the program code into part **3(d)(ii)** in the evidence document.

[1]

(iii) Test the program with the following inputs in the order given:

999999X

1251484

5500212

0033585

9845788

6666666

3258746

8111022

7568557

0012353

Take a screenshot of the output(s).

Save your program.

Copy and paste the screenshot into part **3(d)(iii)** in the evidence document.

[2]